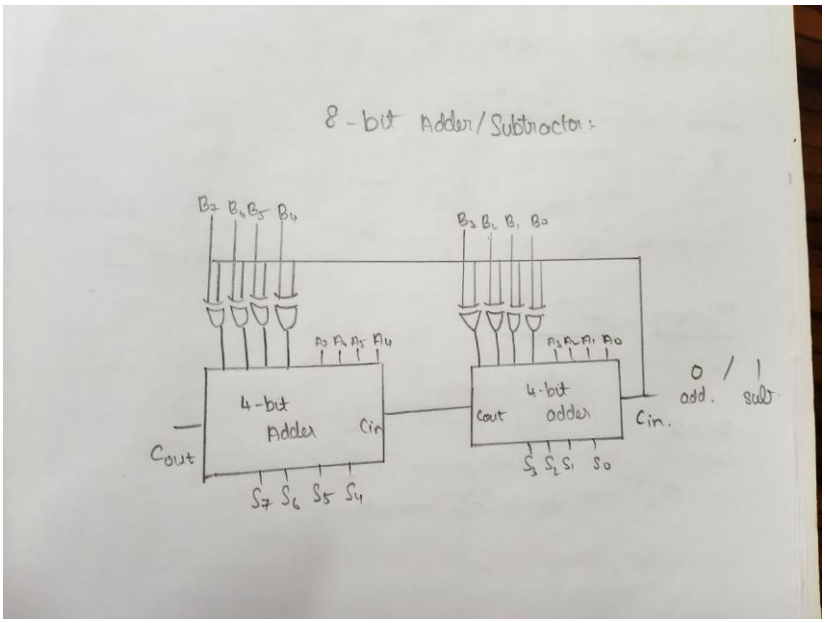
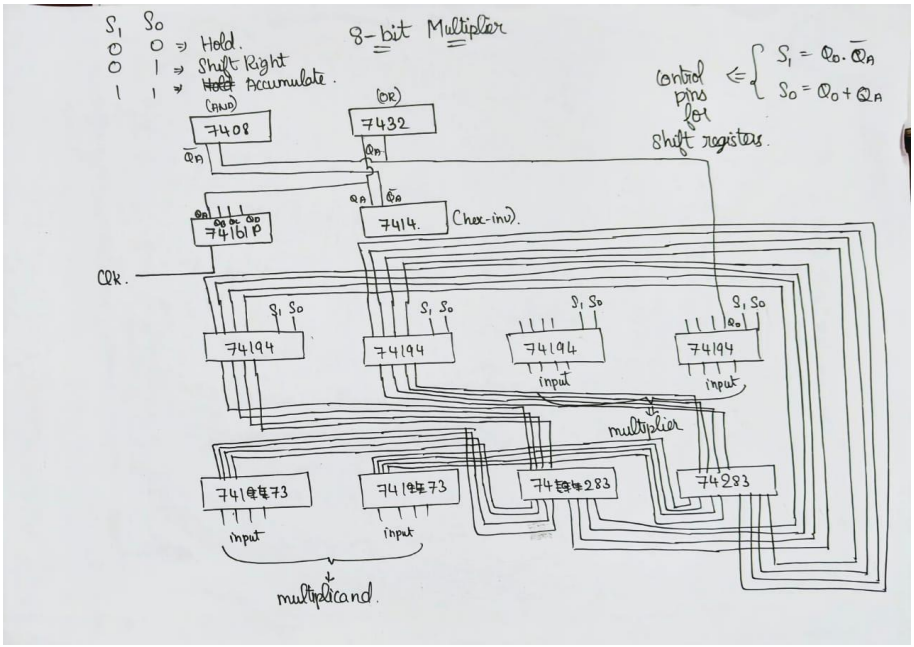
OBJECTIVE:

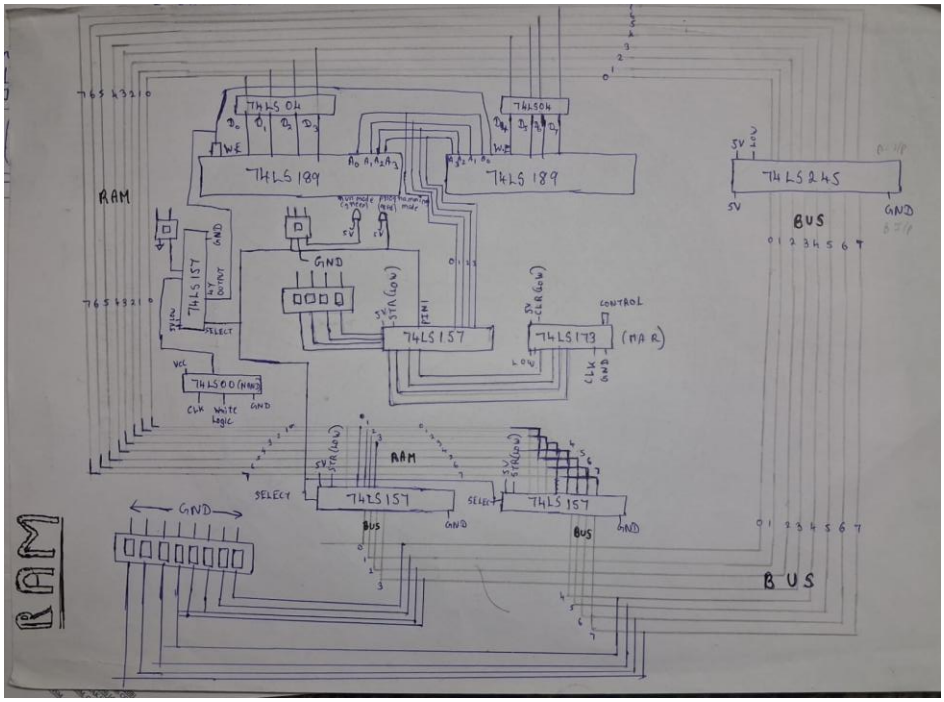
To design and implement a basic 8-bit breadboard computer that can store, add, subtract 2x 8-bit numbers.

BLOCK DIAGRAM:

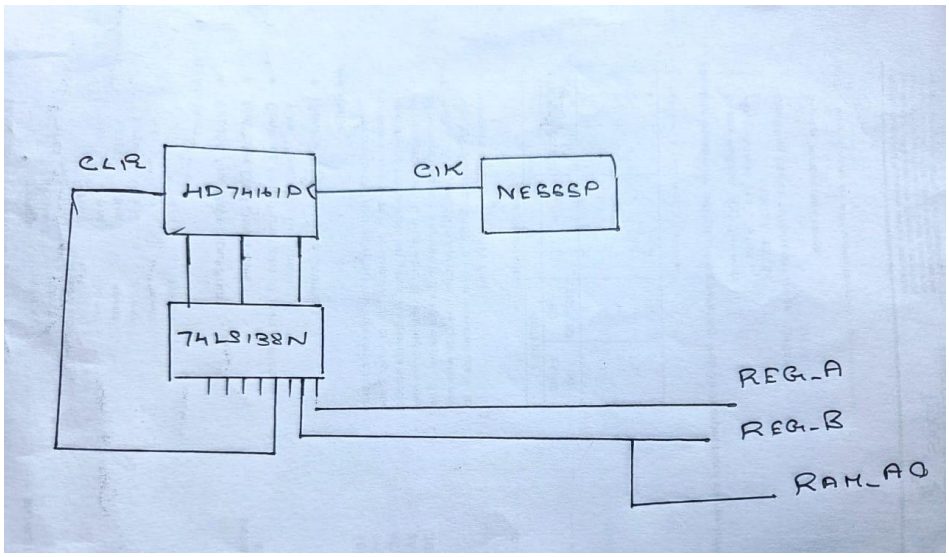


ALU





RAM



Control Unit

COMPONENTS REQUIRED:

Component	Quantity
IC 74LS189 (4-bit memory)	2
IC 74LS194 (Universal Shift Register)	4
IC 74LS157 (2:1 Mux)	4
IC 74LS245 (Tri-state Buffer)	1
IC 74LS283 (4-bit adder)	4
IC 74LS173 (Accumulator/Parallel Register)	3
IC NE555 (Clock Timer IC)	1
IC HD74161P (Counter)	1
IC LS74138N (3:8 Decoder)	1
IC 74LS574 (Registers)	2
IC 74LS14 (Invertor)	1
IC 74LS04 (NOT Gate)	2
IC 74LS00 (NAND Gate)	1
IC 74LS32 (OR Gate)	1
IC 74LS86 (XOR Gate)	2
IC 74LS08 (AND Gate)	1
4-pin DIP	1
8-pin DIP	1
LED's	-
Jumper Wires	-
Breadboard	15
Push Buttons and Toggle Switches	-

PROJECT DESCRIPTION:

The project involves the design and implementation of a **basic 8-bit breadboard computer** using discrete TTL logic ICs. The system is capable of performing fundamental arithmetic operations such as **addition and subtraction of two 8-bit numbers**, along with basic data storage and transfer operations through a common bus architecture.

The computer is built using a **modular design approach**, where each functional unit is implemented separately and interconnected through an 8-bit data bus. The major components of the system include the **Arithmetic Logic Unit (ALU), registers, memory unit, control unit, and clock generation circuit**.

This system follows a simplified von Neumann architecture with a shared bus and hardwired control unit. It is not application-specific, but a general-purpose educational CPU built using discrete logic components. It has a 16 bytes memory and 2 bytes of processing power where it can add, subtract and multiply.

The **Arithmetic Logic Unit (ALU)** is constructed using multiple 4-bit adders (74LS283) cascaded together to perform 8-bit operations. Subtraction is achieved using the **2's complement method**, where

XOR gates (74LS86) conditionally invert the second operand based on the control signal, and an additional carry-in is applied to complete the complement operation. This allows the same hardware to perform both addition and subtraction efficiently.

The system uses multiple **registers (74LS173 and 74LS574)** to store intermediate data such as operands and results. These registers are connected to a shared **8-bit bus**, enabling controlled data transfer between different modules. The bus architecture is managed using **tri-state buffers (74LS245)** and multiplexers (74LS157), ensuring that only one device drives the bus at any given time, thus avoiding conflicts.

A small memory unit is implemented using **74LS189 RAM ICs**, allowing storage and retrieval of 4-bit data words, which are combined to handle 8-bit operations. The memory addressing and data selection are controlled through multiplexers and registers.

The **control unit** is designed using a combination of a binary counter (74161) and a decoder (74LS138), which generates sequential control signals required for step-by-step execution of operations. These control signals coordinate actions such as loading registers, enabling the ALU, reading/writing memory, and controlling data flow across the system.

The **clock signal** required for synchronous operation is generated using a **NE555 timer IC**, which provides a stable pulse to drive the sequential logic components. Manual input is provided through switches, and outputs are visualized using LEDs for easy observation of system behavior.

Overall, the project demonstrates the fundamental working principles of a computer at the hardware level, including **data representation, arithmetic processing, memory access, and control sequencing**, implemented entirely using basic digital logic components on a breadboard.

CONCEPT LEARNED:

1. **Fundamentals of Computer Architecture:**
   Gained a clear understanding of how a basic computer is structured using components such as the ALU, registers, memory, and control unit, and how these blocks interact through a common bus system.

2. **Arithmetic Logic Unit (ALU) Design:**
   Learned how arithmetic operations like addition and subtraction are implemented at the hardware level using combinational logic. Understood the use of **2's complement** for subtraction using XOR gates and carry input.

3. **Register Operations and Data Flow:**
   Understood how registers store and transfer data within the system, and how controlled loading and enabling of registers help in step-by-step execution of instructions.

4. **Bus Architecture and Tri-State Logic:**
   Learned the importance of a shared data bus and how tri-state buffers prevent bus contention by ensuring only one device communicates at a time.

5. **Memory Interfacing:**
   Gained knowledge of how RAM is accessed and controlled, including read/write operations and addressing mechanisms.

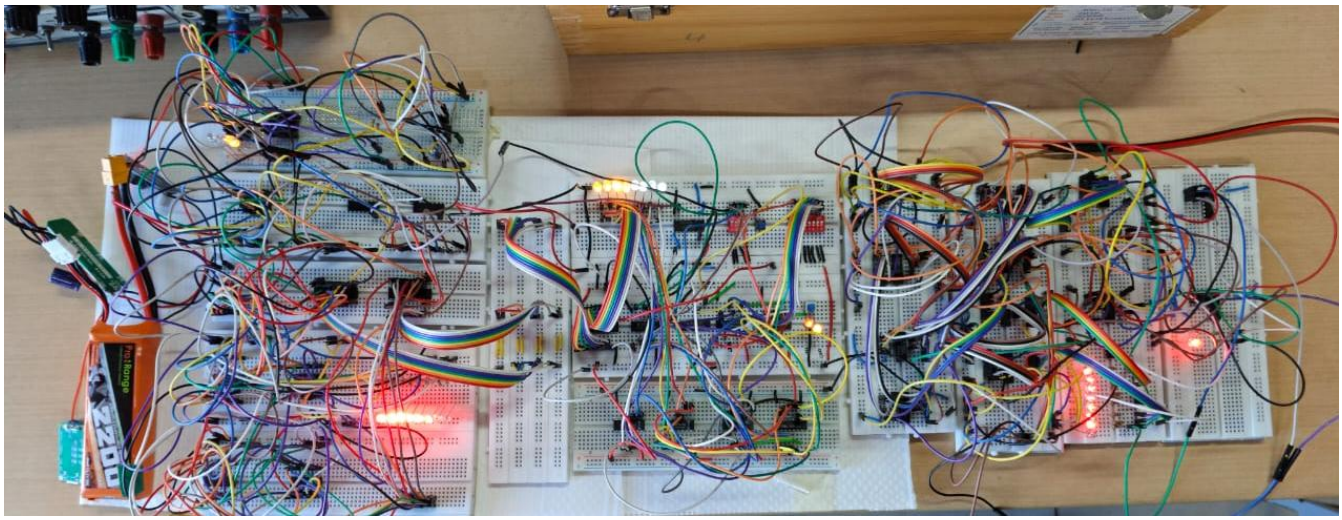
6. **Control Unit and Sequencing:**
   Understood how a control unit generates timing and control signals using counters and decoders to coordinate operations across different modules.

7. **Clock Generation and Synchronization:**
   Learned the role of clock signals in synchronous systems and the importance of timing in ensuring correct operation of sequential circuits.

8. **Practical Hardware Debugging:**
   Developed hands-on skills in identifying and resolving issues such as wiring errors, noise, and unstable signals in real hardware implementations.

9. **System-Level Thinking:**
   Learned how individual digital components come together to form a complete working system, emphasizing modular design and integration.

IMPLEMENTATION:



Youtube link: https://youtu.be/RA9iolRAmug

CHALLENGES FACED:

1. Signal Noise and Wiring Complexity: Due to the large number of jumper wires placed closely on multiple breadboards, significant electrical noise and signal interference were observed. This occasionally led to unstable outputs, especially during clock transitions and

bus operations. The dense wiring also made debugging difficult and increased the chances of loose or incorrect connections.

2. Breadboard Limitations: The use of multiple breadboards made the system physically large and somewhat unstable. Power distribution across boards was not uniform, leading to voltage drops and inconsistent IC behaviour. The setup was also mechanically fragile, making it prone to disconnections during handling.
3. Debugging Complexity: Identifying faults in such a large hardware system was time-consuming. Errors could arise from wiring, IC faults, or incorrect control signals, making it difficult to isolate the exact issue without systematic testing.

APPLICATIONS:

1. Educational Tool for Computer Architecture: This project serves as an excellent learning platform to understand how computers function at the hardware level. It demonstrates key concepts such as registers, ALU operations, memory access, and control sequencing in a hands-on manner.
2. Foundation for Embedded Systems Design: The knowledge gained from designing such a system is directly applicable to embedded systems, where understanding low-level hardware interaction and timing is crucial.
3. Understanding Processor Design: The project provides insight into how modern CPUs are built from basic logic components, helping in the study of microprocessors, FPGA design, and VLSI systems.
4. Digital System Prototyping: The architecture can be extended to design more complex digital systems such as custom processors, controllers, or arithmetic units used in real-world applications.

CONCLUSION:

The 8-bit breadboard computer was successfully designed and implemented using basic TTL logic components. The system was able to perform fundamental operations such as **data storage, addition, and subtraction of 8-bit numbers**, demonstrating the core functionalities of a simple computing system.

Through this project, a deeper understanding of **digital system design, bus architecture, register operations, memory interfacing, and control sequencing** was achieved. The practical challenges faced during implementation, such as noise, synchronization issues, and wiring complexity, provided valuable insights into real-world hardware design constraints.

Overall, this project bridges the gap between theoretical concepts and practical implementation, highlighting how complex computing systems are built from simple logic elements. It also serves as a strong foundation for further exploration in areas such as **embedded systems, processor design, and advanced digital architecture**.